

第 11 章：Model-based RL（Model-based Reinforcement Learning，基于模型的强化学习）、Offline RL（离线强化学习）、Imitation Learning 与 RLHF（Reinforcement Learning from Human Feedback，基于人类反馈的强化学习）

11.1 本章符号说明

符号/缩写	English	中文	含义
RL	Reinforcement Learning	强化学习	通过交互或数据学习策略以最大化长期 reward。
Offline RL	Offline Reinforcement Learning	离线强化学习	只用固定数据集学习策略，不继续和环境交互。
RLHF	Reinforcement Learning from Human Feedback	基于人类反馈的强化学习	用人类偏好训练 reward model，再优化 policy。
$P(s' s, a)$	Transition Probability	状态转移概率	环境 dynamics 的概率形式。
$R(s, a)$	Reward Function	奖励函数	环境或 reward model 给出的奖励。
D	Dataset	数据集	offline RL 或 imitation learning 使用的固定样本集合。
$f_{\varphi}(s, a)$	Dynamics Model	动力学模型	参数 φ 下预测下一状态的模型。
$P_{\varphi}(s' s, a)$	Learned Transition Model	学得转移模型	参数 φ 下对真实环境转移概率 $P(s' s, a)$ 的近似。
$\mu(a s)$	Behavior Policy	行为策略	产生离线数据集 D 的策略，可来自旧系统、人类或多个策略的混合。
d / <code>done</code>	Terminal Indicator	终止指示量	离线 transition 是否到达真正终止状态； $d=1$ 时后续 bootstrap value 为 0。
a_{expert}	Expert Action	专家动作	模仿学习数据中专家在状态 s 给出的动作标签。
θ	Policy Parameters	策略参数	Behavior Cloning 或 RL 优化中当前 policy 的可学习参数。
$\sigma(x)$	Logistic Sigmoid	Logistic Sigmoid 函数	将实数差值映射到 $(0, 1)$ ，在 pairwise reward loss 中表示偏好概率。
$r_{chosen} / r_{rejected}$	Chosen / Rejected Reward Score	被选 / 被拒回答的奖励分数	reward model 给一对回答输出的标量分数。
L_{RM}	Reward-model Loss	奖励模型损失	用 chosen/rejected 分数差训练 pairwise preference 的损失。

符号/缩写	English	中文	含义
π / π_{ref}	Current / Reference Policy	当前 / 参考策略	RLHF 中正在优化的 policy，以及用于约束它不要偏离太远的固定参考模型。
$x / y^+ / y^-$	Prompt / Preferred / Rejected Response	Prompt / 偏好回答 / 被拒回答	DPO 偏好样本中的输入、优选输出和劣选输出。
r_{RM} / r_{total}	RM / Total Reward	奖励模型分数 / 总奖励	PPO 优化时的原始 reward model 分数，以及加入 KL 惩罚后的训练奖励。
β_{KL}	KL-penalty Coefficient	KL 惩罚系数	控制偏离参考策略的惩罚强度。
BC	Behavior Cloning	行为克隆	把专家示范当监督学习来模仿。
IRL	Inverse Reinforcement Learning	逆强化学习	从专家行为反推出 reward function。
SFT	Supervised Fine-Tuning	监督微调	RLHF 中用人工示范数据微调语言模型。
RM	Reward Model	奖励模型	根据偏好数据预测 response 质量的模型。
DPO	Direct Preference Optimization	直接偏好优化	不显式跑 PPO 的偏好优化方法。
KL	Kullback-Leibler Divergence	KL 散度	RLHF 中常用来限制 policy 偏离 reference model。
DQN / PPO / SAC	Deep Q-Network / Proximal Policy Optimization / Soft Actor-Critic	深度 Q 网络 / 近端策略优化 / 软 Actor-Critic	文中作为 model-free 或 RLHF 优化阶段的代表算法。
BCQ	Batch-Constrained deep Q-learning	批约束深度 Q 学习	offline RL 代表算法之一。
CQL	Conservative Q-Learning	保守 Q 学习	通过保守估计 Q 值处理 OOD action 的 offline RL 方法。
IQL	Implicit Q-Learning	隐式 Q 学习	不显式查询 OOD action 的 offline RL 方法。
DT	Decision Transformer	决策 Transformer	把离线轨迹建模成条件序列、根据目标 return 预测动作的方法。
α_{CQL}	CQL Conservative Coefficient	CQL 保守系数	控制 CQL 对未支持动作 Q 值的压低强度。
τ_{IQL}	IQL Expectile Level	IQL 期望分位水平	控制 IQL 的 value 拟合更偏向数据动作中的高价值部分。
β_{IQL}	IQL Advantage Temperature	IQL 优势温度	控制 advantage-weighted behavior cloning 的权重集中程度。
β_{DPO}	DPO Preference Temperature	DPO 偏好温度	控制 policy 相对 reference model 的偏好优化强度。
$L_{CQL} / L_V / L_\pi / L_{DPO}$	CQL / Value / Policy / DPO Loss	CQL / 价值 / 策略 / DPO 损失	本章各离线价值学习、策略学习和偏好优化目标。

符号/缩写	English	中文	含义
OOD	Out-of-Distribution	分布外	数据集中缺少覆盖的状态或动作。

11.2 本章目标

本章补充面试中越来越常见的进阶主题。你需要掌握：

- model-free vs model-based RL。
- offline RL 的核心难点。
- BCQ、CQL、IQL 与 Decision Transformer 分别怎样处理离线数据约束。
- imitation learning 和 behavior cloning。
- inverse reinforcement learning。
- PPO-based RLHF 与 DPO 的数据流和边界。

11.3 本章主线

前 10 章主要讨论在线 model-free 学习。本章换成四类现实约束：能不能学或使用环境模型，能不能继续交互，能不能从专家示范学，能不能用人类偏好定义 reward。Model-based RL、Offline RL、Imitation Learning、RLHF 分别对应这些约束，是面试里从经典 RL 扩展到真实系统和大模型对齐的关键桥梁。

11.4 本章新增概念说明

这一章主题最多，先把容易混的名词集中定义：

名词	中文	先记住什么
model-free RL	无模型强化学习	不显式学习环境模型，直接学 value 或 policy。
model-based RL	基于模型的强化学习	学习或使用环境 dynamics/reward model，再用模型 planning 或 rollout。
dynamics model	动力学模型	预测执行动作后的下一状态，例如 $s_{t+1} \sim f_{\phi}(s_t, a_t)$ 。
planning	规划	在模型里搜索或评估未来动作序列。
model bias	模型偏差	学到的模型不准，规划时会放大误差。
offline RL	离线强化学习	只用固定历史数据集学习策略，不继续探索收集新数据。
behavior policy	行为策略	产生离线数据的旧策略、人类策略或混合策略。
OOD action	分布外动作	历史数据中很少出现或没出现过的动作。
extrapolation error	外推误差	critic 对没数据覆盖的动作估值不可靠。
imitation learning	模仿学习	从专家数据中学习行为。
behavior cloning	行为克隆	把专家动作当监督学习标签来模仿。
compounding error	误差累积	模型犯小错后进入未见状态，后续错误继续放大。

名词	中文	先记住什么
DAgger	数据集聚合	让专家给 learner 实际访问到的状态继续标注，缓解分布偏移。
IRL	逆强化学习	不直接模仿动作，而是从专家行为反推 reward。
reward model	奖励模型	根据偏好或标注预测 response/action 质量的模型。
preference data	偏好数据	人类对两个或多个 response 的好坏比较。
reference model	参考模型	RLHF 中用来约束新 policy 不要偏离太远的原始模型。
reward hacking	奖励黑客	policy 利用 reward model 漏洞拿高分，但真实质量变差。
DPO	直接偏好优化	直接用偏好数据优化 policy，不显式跑 PPO。

11.5 进阶主题总览

11.5.1 四类进阶主题的边界

这一章主题多，先用边界表把它们分开：

主题	核心问题	数据来源	主要风险
Model-based RL	能不能学一个环境模型来 planning	在线交互数据或模拟器	model bias 和长 horizon 误差累积
Offline RL	能不能只用历史数据学新策略	固定日志数据集	OOD action 和 extrapolation error
Imitation Learning	能不能不设计 reward，直接模仿专家	专家状态-动作数据	compounding error
RLHF	能不能用人类偏好构造 reward	prompt-response 偏好比较	reward hacking、偏好噪声、KL 约束

它们共同点是：真实交互昂贵、危险或不充分，因此不能只靠“在线试错 + model-free 更新”解决。

11.5.2 Model-free vs Model-based

Model-free RL 不显式学习环境模型，直接学习 value 或 policy：

- Q-learning。
- DQN。
- PPO。
- SAC。

Model-based RL 显式学习或使用环境模型：

$$P(s' | s, a), R(s, a)$$

然后基于模型进行 planning 或生成模拟数据。

11.6 Model-based RL

11.6.1 Model-based RL 的流程

典型流程：

1. 收集环境交互数据。
2. 学习 dynamics model:

$$s_{t+1} \approx f_{\phi}(s_t, a_t)$$

3. 用模型进行 planning 或 rollout。
4. 根据真实环境反馈继续修正模型。

优点：

- sample efficiency 可能更高。
- 可以通过模型进行规划。
- 在真实交互昂贵的场景中有吸引力。

缺点：

- 模型误差会累积。
- 长 horizon rollout 容易偏离真实环境。
- 高维复杂环境中 dynamics 很难学准。

面试表达：

Model-based RL 的核心优势是样本效率，但主要问题是 model bias。学到的 dynamics model 有误差，规划或 rollout 会放大这些误差，尤其在长时序任务中更明显。

11.7 Offline RL

11.7.1 Offline RL

Offline RL，也叫 batch RL，目标是在不继续和环境交互的情况下，只用固定数据集学习策略。

数据集：

$$D = \{(s, a, r, s', d)\}$$

每条记录包含当前状态 s 、动作 a 、奖励 r 、下一状态 s' 和终止标记 d 。这些数据来自某个 behavior policy $\mu(a|s)$ ，可能是人类、旧系统或混合策略。

与 online RL 的区别：

- online RL 可以探索并收集新数据。
- offline RL 只能使用已有数据。

11.7.2 Offline RL 的难点

核心问题：distribution shift。

学习到的新策略可能选择数据集中很少出现甚至没出现过的动作。此时 Q 函数对这些 out-of-distribution actions 的估计不可靠。

如果 Q 函数错误高估了某些 OOD 动作，policy improvement 会进一步偏向这些动作，导致性能崩溃。

这也叫 extrapolation error。

面试表达：

Offline RL 难在不能通过真实环境纠错。策略可能选择数据分布之外的动作，而 critic 对这些动作的 Q 值没有可靠监督，容易产生过估计并被 policy 利用。

11.7.3 Offline RL 的常见思路

普通 online Q-learning 的 policy improvement 会查询：

$$\max_a Q(s,a)$$

Offline 数据无法验证这个最大值对应的动作是否可靠。不同算法的区别，主要在于怎样避免“从未见过的动作因为估值错误而被选中”：

路线	核心约束	代表方法
约束候选动作	只在 behavior data 支持的动作附近优化	BCQ
保守价值估计	主动压低数据外动作的 Q 值	CQL
只做 in-sample 学习	不对数据外动作显式取最大值	IQL
条件序列建模	不做 Bellman maximization，直接预测动作	Decision Transformer

下面不是要求你背完所有论文细节，而是要看清每种路线究竟把普通 Q-learning 的哪一步改了。

11.7.4 BCQ：把 Policy Improvement 限制在数据支持内

BCQ 是 Batch-Constrained deep Q-learning。它的出发点是：

如果离线数据里几乎没出现某个动作，就不要仅凭一个不可靠的高 Q 值选择它。

典型流程：

1. 从数据集 D 学一个 behavior action generator，近似 $\mu(a/s)$ 。
2. 在给定状态 s 下，从 generator 采样若干数据分布内的候选动作。
3. 可用一个小 perturbation network 对候选动作做有限调整。
4. 只在这些候选动作中选择 Q 值最大的动作。

普通 Q-learning：

在整个动作空间 $\max Q(s, a)$

BCQ：

先生成 behavior-supported candidates

再在候选集合中 $\max Q(s, a)$

BCQ 的收益是减少 OOD action；代价是 learned policy 很难远离 behavior data，也可能无法发现数据覆盖之外的更优策略。

11.7.5 CQL：让没有数据支持的高 Q 付出代价

CQL 是 Conservative Q-Learning。它保留 Bellman regression，但额外加入保守正则。

离散动作下的直观形式：

$$L_{CQL} = L_{Bellman} + \alpha_{CQL} [\log \sum_a \exp Q(s,a) - \mathbb{E}_{a \sim D} [Q(s,a)]]$$

两部分作用：

- $\log \sum_a \exp Q(s,a)$ 会惩罚所有动作中虚高的 Q。
- 数据动作的 $Q(s,a)$ 被加回来，避免把有真实支持的动作同样压得过低。

结果是：

对数据中出现过的动作可以较有信心；对没覆盖的动作宁可低估，也不要让 policy 利用偶然高估。

连续动作无法枚举求和，实践中会从若干动作分布采样近似该保守项。 α_{CQL} 太小可能仍会 OOD 高估，太大则可能过度保守。

11.7.6 IQL：不显式查询 OOD 动作

IQL 是 Implicit Q-Learning。它不在下一状态对任意动作做 $\max_a Q(s',a)$ ，而是只用数据集中的动作学习。

第一步，对数据动作的 $Q(s,a)$ 做 expectile regression (非对称平方回归)，学习一个偏向高价值数据动作的 $V(s)$ ：

$$L_V = \mathbb{E}_{(s,a) \sim D} [\tau_{IQL} - I[Q(s,a) - V(s) < 0] \cdot (Q(s,a) - V(s))^2]$$

当 $\tau_{IQL} > 0.5$ 时， $V(s)$ 更靠近数据集中较高的 Q 值，但仍只看真实数据动作。

第二步，用：

$$y = r + \gamma(1-d)V(s')$$

训练 Q。这里没有对数据外动作取 max。

第三步，用 advantage-weighted behavior cloning 学策略：

$$L_\pi = -\mathbb{E}_{(s,a) \sim D} [\exp(\beta_{IQL}(Q(s,a) - V(s))) \log \pi_\theta(a|s)]$$

数据中 advantage 高的动作获得更大模仿权重，差动作权重更低。

IQL 的主线是：

```
不生成 OOD action 做 max
-> 在数据动作内学习偏高价值的 V
-> 用 V bootstrap Q
-> 加权模仿数据中的好动作
```

11.7.7 Decision Transformer：把 Offline RL 改写成序列预测

Decision Transformer 不显式学习 Bellman Q target，而是把一条轨迹组织成序列：

```
return-to-go, state, action,
return-to-go, state, action, ...
```

训练目标类似监督序列建模：给定历史状态、动作和希望达到的 return-to-go，预测下一个动作。

部署时：

1. 指定目标 return。
2. 输入当前状态和历史轨迹。
3. 模型预测下一动作。
4. 执行动作后, 根据实际 reward 更新剩余 return-to-go。

它的优势:

- 不需要显式做 OOD action 上的 Bellman maximization。
- 可以复用 Transformer 的序列建模能力。

它的限制:

- 本质上仍受离线数据覆盖限制。
- 给出数据从未实现过的高目标 return, 不代表模型真能组合出更优行为。
- 对 reward scale、context length 和轨迹质量敏感。

11.7.8 四种方法的统一比较

方法	是否显式学 Q	怎样避免 OOD 问题	主要代价
BCQ	是	候选动作来自 behavior generator	策略受数据分布限制较强
CQL	是	压低未支持动作的 Q	可能过度保守
IQL	是	不对任意 OOD 动作做 max	依赖数据内存在较好动作
Decision Transformer	否	用条件序列预测替代 Bellman max	依赖轨迹覆盖和目标 return 条件

11.8 Imitation Learning

11.8.1 Imitation Learning

Imitation Learning 从专家数据中学习策略。

专家数据:

$$(s, a_{expert})$$

目标: 让 agent 模仿专家动作。

11.8.2 Behavior Cloning

Behavior Cloning, 简称 BC, 是最简单的 imitation learning。

把专家数据当监督学习:

$$\max_{\theta} \log \pi_{\theta}(a_{expert} | s)$$

或者最小化:

$$L(\theta) = -\log \pi_{\theta}(a_{expert} | s)$$

优点:

- 简单。
- 稳定。

- 不需要 reward。
- 训练像监督学习。

缺点：

- distribution shift。
- compounding error。
- 遇到专家数据中没覆盖的状态时容易出错。

为什么会 compounding error?

训练时模型只见过专家状态分布。部署时一旦模型犯小错，进入专家没覆盖的状态，就更容易继续犯错，误差逐步累积。

11.8.3 DAgger

DAgger 是 Dataset Aggregation。

核心思路：

1. 用当前策略和环境交互。
2. 让专家给当前策略访问到的状态标注正确动作。
3. 把新数据加入数据集。
4. 重新训练策略。

DAgger 通过收集 learner 自己会遇到的状态，缓解 behavior cloning 的分布偏移。

11.8.4 Inverse Reinforcement Learning

IRL 的目标不是直接模仿动作，而是从专家行为中反推 reward function。

直觉：

专家之所以这么行动，是因为它在优化某个隐藏 reward。IRL 希望恢复这个 reward，然后再用 RL 学策略。

适合场景：

- 专家动作可观察。
- reward 很难手工设计。
- 希望学到可迁移的偏好或目标。

缺点：

- 问题不适定：多个 reward 都可能解释同一行为。
- 计算复杂。
- 实践中常被更直接的 imitation 或 preference learning 替代。

11.9 RLHF

11.9.1 RLHF

RLHF 是 Reinforcement Learning from Human Feedback，常用于大语言模型对齐。

典型流程：

1. Pretraining：在大规模语料上预训练基础模型。
2. SFT：用人工示范数据做 supervised fine-tuning。
3. Reward Model：收集人类偏好比较，训练 reward model。
4. RL Optimization：用 PPO 等算法优化 policy，使其获得更高 reward model 分数。

偏好数据形式：

$$(prompt, response_{chosen}, response_{rejected})$$

Reward model 学习：

$$r_{chosen} > r_{rejected}$$

常见 pairwise loss：

$$L_{RM} = -\log \sigma(r_{chosen} - r_{rejected})$$

11.9.2 RLHF 中的 PPO

在 RLHF 中，policy 是语言模型。动作是 token，trajectory 是生成序列。

优化目标通常包含：

- reward model 分数。
- KL penalty，限制新模型不要偏离 reference model 太远。

形式：

$$r_{total} = r_{RM} - \beta_{KL} KL(\pi \| \pi_{ref})$$

为什么需要 KL penalty？

- 防止模型为了骗 reward model 而偏离语言质量。
- 保持和 SFT/reference model 的行为接近。
- 提高训练稳定性。

11.9.3 RLHF 的常见问题

- Reward hacking：模型找到 reward model 漏洞。
- Preference data 噪声。
- Reward model 泛化问题。
- PPO 训练不稳定。
- 对齐目标难以完全由标注偏好表达。

11.9.4 DPO：为什么可以不显式训练 Reward Model 和 PPO

DPO 是 Direct Preference Optimization。它使用同样的 pairwise preference data：

$$(x, y^+, y^-)$$

其中 x 是 prompt， y^+ 是 preferred response， y^- 是 rejected response。

DPO loss 的常见形式：

$$L_{DPO} = -\log \sigma(\beta_{DPO} [\log \pi_{\theta}(y^+ | x) - \log \pi_{ref}(y^+ | x) - \log \pi_{\theta}(y^- | x) + \log \pi_{ref}(y^- | x)])$$

先看括号里的两项：

$$\log \pi_{\theta}(y^+ | x) / \{\pi_{ref}(y^+ | x)\} - \log \pi_{\theta}(y^- | x) / \{\pi_{ref}(y^- | x)\}$$

它要求当前 policy 相对 reference model：

- 更多提高 preferred response 的概率。
- 更多降低 rejected response 的概率。

DPO 来自 KL-regularized preference RL 的解析关系：在特定假设下，可以消去显式 reward function，把 reward difference 改写成 policy 与 reference policy 的 log-ratio difference。

因此经典两条路线是：

PP0-based RLHF：

```
preference data -> reward model
-> online sample responses
-> PP0 maximize RM reward with KL constraint
```

DPO：

```
preference pairs + reference policy
-> directly optimize pairwise classification objective
```

DPO 的优点：

- 不需要单独部署 reward model 做在线打分。
- 不需要 rollout + Critic + PPO 的复杂训练循环。
- 训练形态接近 supervised fine-tuning。

DPO 没有自动解决的问题：

- 仍依赖偏好数据质量和覆盖。
- 主要是 offline preference optimization，不能自动探索数据之外的新行为。
- Reference model 和 β_{DPO} 会影响优化强度。
- 它简化了优化流程，不代表“偏好对齐已经没有 reward 或 RL 的概念来源”。

11.10 本章例子：四类高级 RL 场景

11.10.1 例子 1：Model-based RL 做机器人规划

机器人先学习一个 dynamics model：

$$P_{\phi}(s' | s, a)$$

然后在模型里模拟不同动作序列：

```
动作序列 1 -> 预测会撞墙
动作序列 2 -> 预测能到目标点
动作序列 3 -> 预测路径更短但风险高
```

再选择预测回报最高的动作序列执行。

11.10.2 例子 2：Offline RL 做历史推荐日志

公司已经有大量推荐日志：

用户状态、展示内容、点击/停留/购买

Offline RL 希望只用这些历史数据学习新策略。但难点是新策略可能推荐历史日志里很少出现的内容，价值估计容易外推错误。

所以 offline RL 常需要保守约束：不要让策略偏离数据分布太远。

11.10.3 例子 3：Imitation Learning 做驾驶

Behavior Cloning 可以用人类驾驶数据训练：

输入：摄像头、速度、导航

输出：方向盘角度、油门、刹车

问题是模型一旦偏离人类数据分布，就可能进入训练集中没见过的状态。DAgger 的思路是让专家在模型实际访问到的状态上继续标注，补齐分布偏移。

11.10.4 例子 4：RLHF 做聊天模型

RLHF pipeline：

1. 用 SFT 让模型学会基本回答格式。
2. 收集同一个 prompt 下多个回答的人类偏好。
3. 训练 reward model。
4. 用 PPO 等方法优化策略，同时用 KL 限制模型偏离参考模型。

面试表达：

Model-based、Offline、Imitation、RLHF 的共同点是都在解决“真实交互成本高或风险大”的问题：要么学模型模拟，要么复用历史数据，要么模仿专家，要么用人类偏好构造 reward。

11.11 本章面试高频问题

11.11.1 Model-free 和 model-based RL 区别？

Model-free 不显式学习环境模型，直接学习 policy 或 value。Model-based 学习或使用环境 dynamics 和 reward model，并基于模型 planning 或生成数据。

11.11.2 Model-based RL 的主要问题是什么？

模型误差。学到的 dynamics model 不可能完全准确，长 horizon planning 会累积误差，导致策略利用模型漏洞而在真实环境中表现差。

11.11.3 Offline RL 为什么难？

因为只能使用固定数据集，不能通过环境探索纠错。策略可能选择数据分布之外的动作，而 Q 函数对这些动作估计不可靠，容易产生 extrapolation error。

11.11.4 Behavior Cloning 的问题？

BC 把模仿学习当监督学习，简单稳定，但有 distribution shift 和 compounding error。模型部署后遇到专家数据没覆盖的状态，错误会逐步累积。

11.11.5 RLHF 的基本流程？

先预训练语言模型，再用示范数据 SFT，然后用人类偏好训练 reward model，最后用 PPO 等 RL 算法在 KL 约束下优化模型输出。

11.12 本章练习

1. 用一句话区分 online RL 和 offline RL。
2. 解释 offline RL 中的 extrapolation error。
3. 对比 Behavior Cloning 和 RL。
4. 画出 RLHF 的四阶段流程。
5. 分别说明 BCQ、CQL、IQL 和 Decision Transformer 怎样避免直接利用 OOD action。
6. 对比 PPO-based RLHF 和 DPO 的训练数据流。

▼ 参考答案（点击展开）

1. **Online RL vs Offline RL**：Online RL 可以继续与环境交互并通过探索改变数据分布；Offline RL 只能使用固定数据集学习，不能在线试错，因此必须控制对数据分布外动作的错误估值。
2. **Extrapolation error**：数据集中很少或从未出现的 (s,a) 上，function approximator 的 Q 估计缺少监督。Policy improvement 又会偏向选择被偶然高估的分布外动作，随后 bootstrap 把错误继续传播，造成性能崩溃。常见缓解思路是保守 Q 估计、行为约束或只在数据支持区域内优化。
3. **Behavior Cloning vs RL**：Behavior Cloning 把专家 (s,a) 当监督数据，直接最大化专家 action 的 likelihood，简单稳定但会有 covariate shift，也无法主动超过示范；RL 根据 reward 和环境交互优化长期 return，需要探索和 credit assignment，但可以发现示范中没有的更优行为。Imitation Learning 还包括 DAGger、IRL 等，不等同于只做 BC。
4. **经典 RLHF 四阶段**：

```
pretrained language model
-> supervised fine-tuning on demonstrations
-> preference data + reward model training
-> policy optimization with reward and KL constraint
```

第四阶段经典做法是 PPO；实际系统也可能采用 DPO 等直接偏好优化方法，此时不显式训练 reward model 并运行在线 PPO。

5. **四类 Offline 方法**：BCQ 先从 behavior-supported generator 产生候选动作，只在候选内取高 Q；CQL 对所有动作的虚高 Q 加保守惩罚，同时保留数据动作价值；IQL 只在数据动作上做 expectile value regression，再用 advantage-weighted behavior cloning 学 policy，不显式查询 OOD action；Decision Transformer 不做 Bellman maximization，而是根据历史轨迹和目标 return 监督预测下一动作。
6. **PPO-based RLHF vs DPO**：PPO 路线先用偏好对训练 reward model，再让当前 policy 在线生成 response，由 reward model 打分，结合 reference KL 后用 Actor-Critic/PPO 更新。DPO 直接读取固定的 (x,y^+,y^-) 偏好对，优化 current/reference log-ratio difference，不显式训练 reward model、Critic 或执行 PPO rollout；它更简单，但仍受离线偏好数据覆盖限制。